DM587

Scientific Programming

Lecture 03: Errors, files and formats

Sai Ganesh Nagarajan

26 October 2026

Assignment 0

Once you submit your GitLab details, the instructor will assign your private project.

Sign in to SDU GitLab. Once assigned, your project will be available at:

```
dm587-2026/YOUR_USERNAME-repo
```

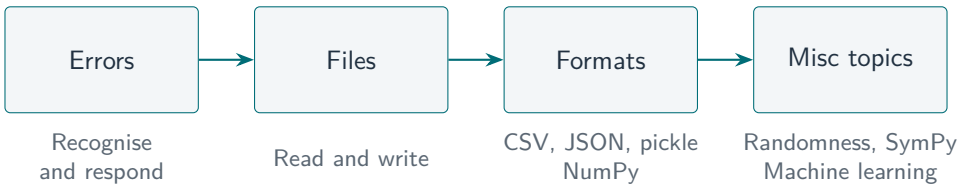
Replace `YOUR_USERNAME` with your GitLab username.

Follow the submission guide on the course webpage and try submitting before the deadline.

Deadline: 30 October 2026, 11:59 pm local time.

If you cannot find or open your assigned project, contact the instructor.

Today



A running example: temperature readings from a small experiment.

01

Errors

What went wrong, and what should happen next?

Section reading

ACME Labs: Exceptions and File I/O, Exceptions

Python tutorial, §8

Debugging



Filipe Fortes, 2013. Original author and context Tweet image

Bugs

Syntax	**Runtime**	**Logic**
Missing colon	Division by zero: $1/0$	Divide by the wrong count
Python cannot parse the code	An exception interrupts execution	The program runs but the answer is wrong

A program that finishes still needs its results checked.

Exceptions

An **exception** is Python's way of reporting a problem while a program is running.

```
denominator = 0
result = 1 / denominator
print(result)
print("Finished")
```

OUTPUT

```
ZeroDivisionError:
division by zero
```

- The division cannot be completed, so Python raises an exception.
- The remaining lines here are skipped.
- Without a matching handler, the error propagates and may stop the program.

Handling an expected error lets us explain the problem and decide what to do next.

Exception types

Exception	Typical cause
`ValueError`	`float("warm")`
`TypeError`	`"12" + 3`
`ZeroDivisionError`	`1 / 0`
`IndexError` / `KeyError`	Missing list position / dictionary key
`OSError`	File cannot be opened or written

Catch the type you expect; `except ValueError` does not catch every error.

try / except

An exception occurs

```
denominator = 0
try:
    result = 1 / denominator
    print(result)
except ZeroDivisionError:
    print("Cannot divide by zero.")
print("Finished")
```

OUTPUT

```
Cannot divide by zero.
Finished
```

No exception occurs

```
denominator = 2
try:
    result = 1 / denominator
    print(result)
except ZeroDivisionError:
    print("Cannot divide by zero.")
print("Finished")
```

OUTPUT

```
0.5
Finished
```

try: run normally. **except**: run only for a matching exception.

Python or a package may raise the error; we decide how we want our code to respond.

finally

No matching handler

```
try:
    result = 1 / 0
except ValueError:
    print("Invalid value")
print("Finished")
```

PRINTED OUTPUT

Nothing is printed.

Cleanup with finally

```
try:
    result = 1 / 0
except ValueError:
    print("Invalid value")
finally:
    print("Cleanup")
print("Finished")
```

OUTPUT

```
Cleanup
```

`ValueError` does not match `ZeroDivisionError`; neither example prints `Finished`.

Use `finally` for cleanup on either outcome. It runs before an uncaught exception propagates.

Examples: close a file, close a database connection, remove a temporary file.

Tracebacks

```
def read_temperature(text):
    return float(text)

read_temperature("warm")
```

```
ValueError:  could not convert string to float:  'warm'
```

- Start with the last line: the exception type and message.
- Look just above it for the failing line; follow the calls back if needed.

Successful conversion

```
text = "12.5"
try:
    value = float(text)
except ValueError:
    print("Enter a number.")
else:
    print("Reading:", value)
```

With `text = "12.5"`

Conversion succeeds; `else` runs.

```
OUTPUT
Reading:  12.5
```

With `text = "warm"`

`except` runs; `else` is skipped.

```
OUTPUT
Enter a number.
```

`else` runs if no exception is raised inside `try`.

Use the value only after successful conversion; otherwise, explain the problem.

raise / except

Function: report the problem

```
def mean(values):
    if not values:
        raise ValueError(
            "No readings supplied")
    return sum(values) / len(values)
```

`raise` leaves the function and signals the invalid input to its caller.

Caller: choose the response

```
try:
    answer = mean([])
except ValueError:
    print("Please supply readings.")
```

OUTPUT

```
Please supply readings.
```

Handle it inside the function if it can recover meaningfully.

An empty list has no mean; returning 0 would hide the problem. Let the caller decide whether to ask again, skip the calculation or stop.

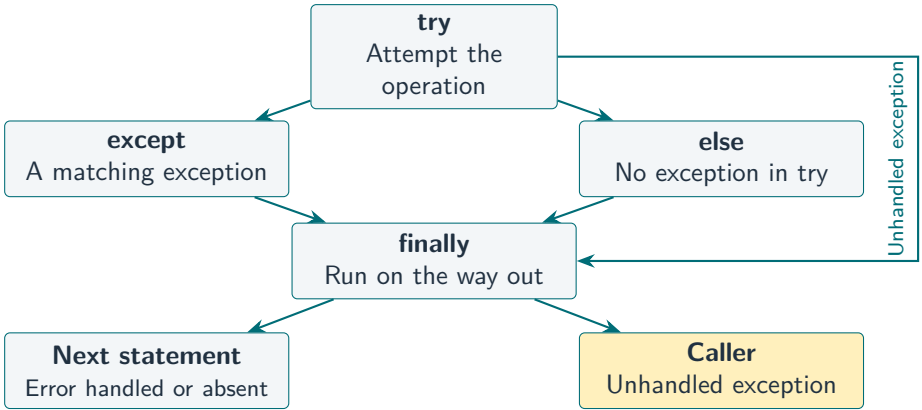
`raise` reports a problem; `try/except` decides how to respond.

Propagation



With no matching handler, execution stops and Python prints a traceback.

Control flow



Continue after the statement only if no exception remains unhandled.

If no caller catches the original exception, Python prints a traceback and execution stops.

Prediction

```
try:
    print("A")
    value = float("warm")
    print("B")
except ValueError:
    print("C")
print("D")
```

Try it

Which letters are printed, and in what order?

Answer

OUTPUT

```
A
C
D
```

- `A` prints before the failing conversion.
- The exception skips `B` and enters the matching handler.
- After the handler, execution continues with `D`.

Prediction

```
text = "2"
try:
    print("A")
    value = int(text)
    print("B")
    result = 1 / value
except ZeroDivisionError:
    print("C")
else:
    print("D")
finally:
    print("E")
print("F")
```

Change only the first line.

Predict the letters printed, in order, for each input:

1. `text = "2"`
2. `text = "0"`
3. `text = "warm"`

Which handler runs?

When does `else` run?

Is `F` reached?

Answer

Input	Path	Printed letters
`"2"`	No exception: `else`	`A B D E F`
`"0"`	`ZeroDivisionError` handler	`A B C E F`
`"warm"`	Unhandled `ValueError`	`A E`

- `E` prints from `finally` in all three cases.
- With `"warm"`, conversion fails before `B`; `finally` prints `E`, then the exception propagates.

For `"warm"`, `F` is not printed. Without a caller handling the error, Python prints a traceback and stops.

Assertions

Assertions check expected results and internal assumptions, helping expose **logical errors**.

```
assert condition
assert condition, "Optional failure message"
```

```
def square(x):
    return x * x

assert square(-3) == 9, "Expected 9"
```

Condition

`square(-3) == 9` is checked.

Optional message

The text after the comma explains a failed check. It is not another condition.

True: continue silently. False: raise `AssertionError` with the message.

Where to check

```python
def square(x):
    return x * x

if __name__ == "__main__":
    assert square(3) == 9
    assert square(-3) == 9, "Expected 9"
    print("Checks passed.")
```

OUTPUT

```
Checks passed.
```

Assertions can go:

- After a calculation or function call.
- Inside a function, to check an internal assumption.
- In separate testing code.

The `__main__` guard is optional. Here it runs these checks when the file is executed directly, and skips them on import.

Place a check where its expected condition should hold.

Checks

Use	Example	Purpose
`assert`	`square(-3)` returns 9	Check our program
`if` + `raise`	No readings supplied	Reject invalid input

Assertions can be disabled with `python -O`. Use ordinary checks for required input validation.

For approximate floating-point results, check a tolerance rather than exact equality.

Good habits

Keep try small
Catch errors from the operation you expect to fail.

Catch specific errors
Avoid hiding an unrelated bug.

Say what failed
Include the input or filename when useful.

Recover deliberately
Retry, skip or stop; do not silently ignore.

02

Files

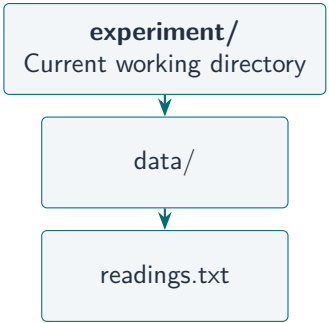
From stored text to useful data.

Section reading

ACME Labs: Exceptions and File I/O, File Input/Output

Python tutorial, §7.2

Paths



Relative path

`data/readings.txt`
Start from the current working directory.

Absolute path

Start from the filesystem root; the full path depends on the machine.

`os.getcwd()` shows the working directory.

Opening files

`open()` opens the file. `with` closes it automatically when the indented block ends.

Normal completion

```
with open("readings.txt", "r") as f:
    text = f.read()  # File open
# File closed automatically here
```

No explicit `f.close()` is needed.

Error inside the block

```
try:
    with open("readings.txt", "r") as f:
        value = float(f.readline())
except ValueError:
    print("Invalid number")
```

If the first line is `"warm"`, conversion fails. The file closes **before** the handler prints.

Closing releases the open file resource; it does not delete the file or its contents.

`with` handles closing the file. `except` handles the error.

Modes

Mode	Purpose	Existing contents
`"r"`	Read text	Kept; file must exist
`"w"`	Write text	**Erased when opened**
`"a"`	Append text	Kept; new text goes at the end
`"rb"`	Read binary data	Kept; file must exist
`"wb"`	Write binary data	**Erased when opened**

Same file: `letters.txt` contains `ABC`, with no newline.

```
with open("letters.txt", "r",
          encoding="utf-8") as f:
    print(repr(f.read()))
```

OUTPUT

```
'ABC'
```

```
with open("letters.txt", "rb") as f:
    data = f.read()
print(repr(data), data.hex())
```

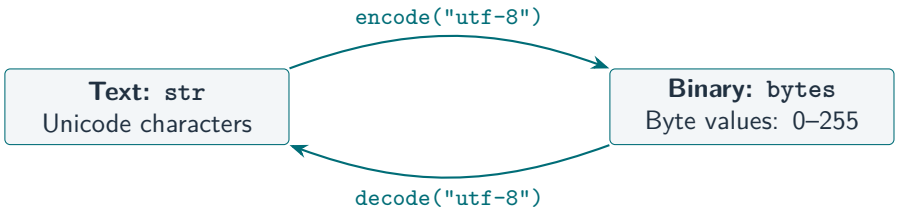
OUTPUT

```
b'ABC' 414243
```

`hex()` displays two hexadecimal digits per byte: `41 42 43`.

`A`, `B`, `C` correspond to `0x41`, `0x42`, `0x43` (decimal 65, 66, 67).

Text and bytes



Text to bytes

```
text = "ABC"
data = text.encode("utf-8")
print(repr(data), data.hex())
```

```
OUTPUT  b'ABC' 414243
```

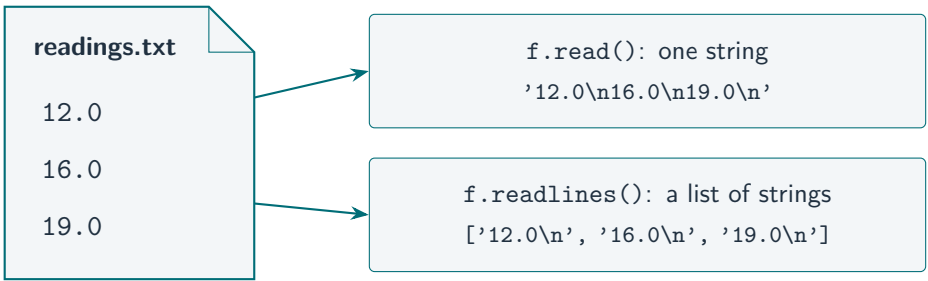
Bytes to text

```
text_again = data.decode("utf-8")
print(repr(text_again))
```

```
OUTPUT  'ABC'
```

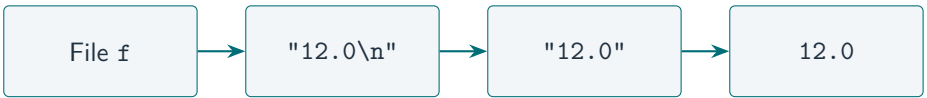
`f` is a file object. `f.read()` returns decoded `str` in text mode, or `bytes` in binary mode. `encoding="utf-8"` selects UTF-8 decoding; omitting it uses Python's default.

Reading



Each example starts with a freshly opened file. `\n` is a newline.

Parsing




```
with open("readings.txt", "r", encoding="utf-8") as f:
    values = [float(line.strip()) for line in f]
```

For this file, `values` becomes `[12.0, 16.0, 19.0]`. A blank line or `"warm"` cannot be converted to a number: `float()` raises `ValueError`. To continue, handle that error and decide whether to skip the line or ask for corrected data.

Writing

```
with open("summary.txt", "w", encoding="utf-8") as f:
    f.write("Mean: ")
    f.write("16.0\n")
```



write() adds exactly the supplied text. Add \n when you want a new line.

Prediction

Before: `notes.txt` contains `old` followed by a newline.

A

```
with open("notes.txt", "w") as f:
    f.write("new\n")
```

B (start again from the same original file)

```
with open("notes.txt", "a") as f:
    f.write("new\n")
```

Try it

What is in the file after A? What is in it after B?

Answer

A: write

```
OUTPUT
new
```

B: append

```
OUTPUT
old
new
```

Choose the mode before opening: w replaces, a extends.

03

Formats

The same data can be stored in different ways.

Section reading

Python library reference: csv
Python tutorial, §7.2.2: JSON
Python library reference: pickle

CSV

Comma-Separated Values: a plain-text format for a table.

readings.csv

```
day,morning,afternoon
Monday,12,19
Tuesday,11,18
```

Day	Morning	Afternoon
Monday	12	19
Tuesday	11	18

Each row is a record; commas separate its fields. The first row often gives column names.

CSV stores text. Convert numerical fields explicitly after reading.

CSV in Python

`import csv`

Same file: header `day,morning,afternoon`, then `Monday,12,19`.

`csv.reader`: lists

```
with open("readings.csv", "r",
          newline="", encoding="utf-8") as f:
    rows = list(csv.reader(f))
print(rows[0])
```

OUTPUT

```
['day', 'morning', 'afternoon']
```

The header remains the first row.

First morning value: `rows[1][1]` → `"12"`.

`csv.DictReader`: dictionaries

```
with open("readings.csv", "r",
          newline="", encoding="utf-8") as f:
    rows = list(csv.DictReader(f))
print(rows[0])
```

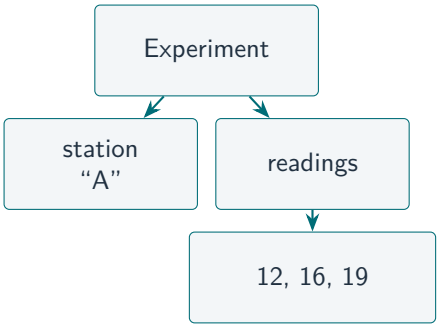
OUTPUT

```
{'day': 'Monday', 'morning': '12',
'afternoon': '19'}
```

The header supplies the keys; `rows[0]` is the first data row.

`rows[0]["morning"]` → `"12"`.

JSON



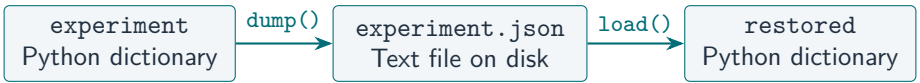

```
{
  "station": "A",
  "readings": [12, 16, 19],
  "checked": true,
  "note": null
}
```

A text format for nested data, used by many languages.

JSON is a file format. A dict or list is a Python data type.

JSON files

```
import json
experiment = {"station": "A", "readings": [12, 16, 19]}
```



Save

```
with open("experiment.json", "w",
          encoding="utf-8") as f:
    json.dump(experiment, f, indent=2)
```

`indent=2` makes the file easier to read.

Load

```
with open("experiment.json", "r",
          encoding="utf-8") as f:
    restored = json.load(f)
print(restored)
```

OUTPUT

```
{'station':  'A', 'readings':  [12, 16, 19]}
```

Prediction

```
import json

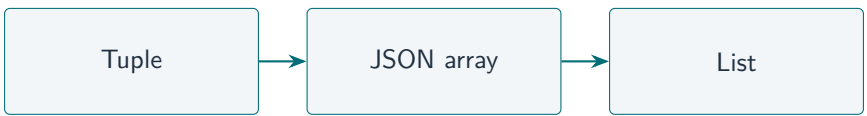
point = (3, 4)
with open("point.json", "w", encoding="utf-8") as f:
    json.dump(point, f)
with open("point.json", "r", encoding="utf-8") as f:
    restored = json.load(f)
print(restored)
print(type(restored))
```

Try it

Do we get a tuple back?

Answer

```
OUTPUT
[3, 4]
<class 'list'>
```



A round trip preserves only the distinctions the format can represent.

JSON recap

Python: before saving	Stored as JSON	Python: after loading
Dictionary `{"x": 3, "y": 4}`	Object `{"x": 3, "y": 4}`	Dictionary `{"x": 3, "y": 4}`
List `[3, 4]`	Array `[3, 4]`	List `[3, 4]`
Tuple `(3, 4)`	Array `[3, 4]`	**List** `[3, 4]`

JSON has no tuple type: lists and tuples are both saved as arrays, which Python loads as lists.

Pickle



- Preserves more Python types, including many class instances.
- Useful for Python checkpoints; less portable across environments.
- Class definitions usually need to be available when loading.

Trusted files only

Loading a pickle can execute code. Never unpickle an untrusted file.

Pickle in Python

```
import pickle

with open("checkpoint.pkl", "wb") as f:
    pickle.dump(experiment, f)

with open("checkpoint.pkl", "rb") as f:
    restored = pickle.load(f)
```

Binary mode: `wb` to save, `rb` to load.

Use JSON or CSV when people or other languages need to read the data.

NumPy files

$$T = \begin{pmatrix} 12 & 16 & 19 \\ 11 & 15 & 18 \end{pmatrix}$$

Shape $(2, 3)$; integer entries

Text: `.txt` or numeric `.csv`

`np.savetxt` / `np.loadtxt`
Human-readable numbers; choose a delimiter and precision.

Binary: `.npy`

`np.save` / `np.load`
Preserves array values, shape and dtype.

Several arrays: `.npz`

`np.savez` / `np.load`

Use loadtxt for simple numeric tables; use csv or pandas for mixed records.

Array round trips

```
np.savetxt("T.csv", T,
           delimiter=",", fmt="%d")
A = np.loadtxt("T.csv",
               delimiter=",")
```

```
np.save("T.npy", T,
        allow_pickle=False)
B = np.load("T.npy",
            allow_pickle=False)
```

Text file

```
12,16,19
11,15,18
```

`fmt="%d"` writes integers here. A is floating-point by default.

Binary file

```
B.shape == T.shape
B.dtype == T.dtype
```

B has the same values as T.

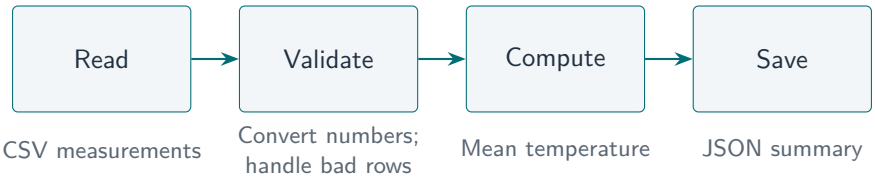
These examples use an ordinary numeric array, not an object array.

Choosing a format

Format	Useful for	Remember
CSV	Tabular data: rows and columns	Convert fields to the right types
JSON	Settings and nested records	Basic types; string keys
NumPy	Numerical arrays	Keeps shape and dtype
Pickle	Trusted Python checkpoints	Python-specific; trust and compatibility

Choose for the data and the reader, not just convenience.

Putting it together



- Missing file: explain which path could not be opened.
- Invalid reading: identify the row and decide whether to skip or stop.
- No usable readings: report the problem rather than inventing a mean.

The try-it notebook contains the complete example.

04

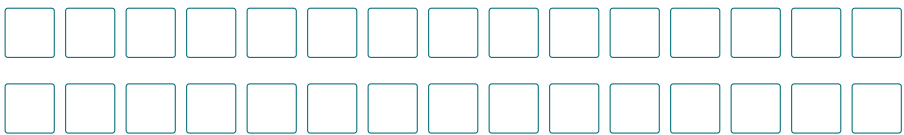
Misc topics

Randomness, symbolic calculations and machine learning.

Section reading

NumPy: Random sampling
SymPy tutorial: Calculus
TensorFlow: Gradients and automatic differentiation

Make it random



Thirty flips of a fair coin: write your own sequence.

- Take one minute. Use only 0 and 1; no coin or computer.
- Compare with a neighbour: count the 1s and find the longest repeated run.
- Does a long run of 1s make the next 0 more likely?

Randomness

A computer-generated sequence

```
0 1 1 0 0 1 0 1 0 0 1 1 1 1 1 1 1 0 1 0 1 0 0 1 1 1 0 1 1 0
```

- People often avoid long runs and alternate too regularly.
- Independent fair flips can contain clusters. The next flip still has probability $1/2$ for either outcome.
- Compare several sequences, rather than judging from one short sample.

Every particular length-30 sequence has the same probability, 2^{-30}.

Reading: human-generated randomness, 2024 study

Random numbers

```
import numpy as np
rng = np.random.default_rng(42)

bits = rng.integers(0, 2, size=30)
u = rng.random(5)
z = rng.normal(0, 1, size=5)
```

`bits`	30 integers: 0 or 1; the upper bound 2 is excluded
`u`	5 uniform samples in $[0, 1)$
`z`	5 normal samples with mean 0 and standard deviation 1

Use `size=(3,4)` to generate a whole array.

Seeds

```
a = np.random.default_rng(42)
b = np.random.default_rng(42)

print(a.integers(0, 2, 6))
print(b.integers(0, 2, 6))
```

OUTPUT

```
[0 1 1 0 0 1]
[0 1 1 0 0 1]
```

A seed sets the starting state.

The generator follows a deterministic algorithm: these are pseudorandom numbers.

Same seed, same calls and environment: repeatable results.

Further calls advance the state. Recreating the generator with the same seed starts over.

Create the generator once per experiment. Omit the seed for a fresh start. Record NumPy's version when exact reproducibility matters; a seed does not make a sample "more random".

SymPy

```
import sympy as sp
x = sp.symbols("x")
f = (x**2 + 1)**2
df = sp.diff(f, x)
print(df)
print(df.subs(x, 3))
```

OUTPUT

```
4*x*(x**2 + 1)
120
```

Work with expressions

x is a symbolic variable.

`diff` produces a derivative expression:

$$f'(x) = 4x(x^2 + 1).$$

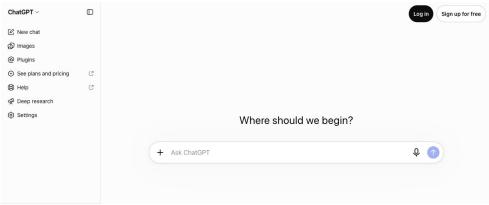
`subs` evaluates it at $x = 3$.

Other tools include `integrate`, `solve` and `simplify`.

Useful for checking algebra and connecting exact expressions to numerical work.

Modern ML

ChatGPT



Text and tokens $\longrightarrow$ a response

Cat or dog?





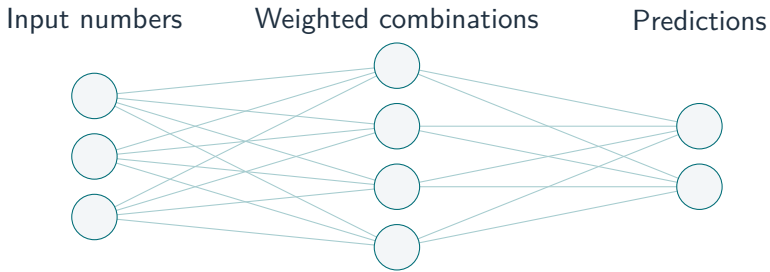
Cat Dog

Pixels $\longrightarrow$ a predicted class

Neural networks learn patterns by adjusting numerical weights.

ChatGPT screenshot: OpenAI. Photos: TensorFlow Datasets, Cats vs Dogs. Labels shown are dataset labels, not model outputs.

Neural networks



Edges carry weights; layers also apply nonlinear functions.

one layer: $h = \sigma(Wx + b)$

Learning means finding weights that reduce a loss: a measure of prediction error.

Gradients

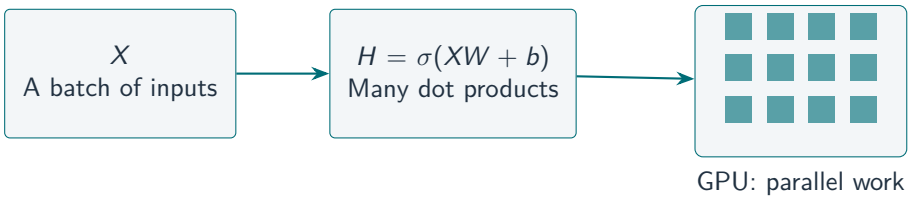


$$W \leftarrow W - \eta \nabla_W L \qquad \text{(a gradient-descent step)}$$

- The gradient tells us how changing weights affects the loss; η sets the step size.
- TensorFlow and PyTorch compute gradients of supported operations automatically.

For now: understand what a gradient is used for; implementing autodiff can wait.

Matrices and GPUs

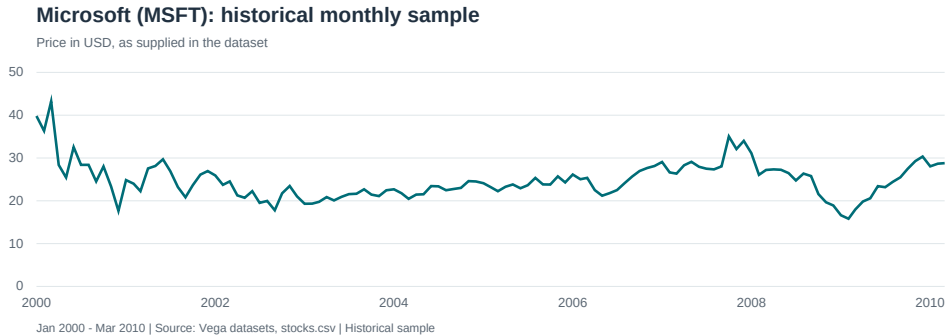


- Matrix products expose many multiply-and-add operations that can run in parallel.
- Batching and reusing data help large computations use GPUs efficiently.
- TensorFlow / PyTorch connect Python to optimised CPU and GPU kernels.

Ordinary NumPy arrays normally run on the CPU. GPUs are not faster for every small task.

Next class

pandas / Visualization / Time series



What changes when we look at prices, returns or a rolling average?

A real-data preview; the planned time-series activity will use weather data. Data source

Try it

- Run the companion notebook from top to bottom.
- Change a reading, add a bad row and inspect the saved files.
- Predict what happens before running each example.

Readings

ACME Labs: Exceptions and File I/O

Python tutorial, §8 §7.2 csv json pickle NumPy I/O

Examples in this lecture were prepared for the course, following these references.